ICT Express xxx (xxxx) xxx

Contents lists available at ScienceDirect

ICT Express

journal homepage: www.elsevier.com/locate/icte

Malicious encrypted traffic identification model based on flow space-time features☆

Shi Dong *, Jiayin Zang, Khushnood Abbas

School of Computer Science and Technology, Zhoukou Normal University, Zhoukou, 466001, China

ARTICLE INFO

Keywords:
Deep learning
Network security
Malicious encrypted traffic detection
Convolutional neural network
BiLSTM

ABSTRACT

The rapid proliferation of encrypted traffic in modern networks presents significant challenges for security monitoring and threat detection. While encryption enhances privacy, it also provides cover for malicious activities, making traditional deep packet inspection ineffective. To address this challenge, this paper proposes an interpretable and efficient deep learning framework for malicious encrypted traffic identification. Our model integrates 1D-CNN, BiLSTM, and a lightweight self-attention mechanism to capture both spatial and temporal characteristics of network flows while providing explainable predictions through attention visualization. Unlike computationally intensive Transformer-based approaches, our method achieves an optimal balance between accuracy and efficiency, making it suitable for real-world deployment. Extensive experiments on benchmark datasets (USTC-TFC2016 and CICIDS-2017) demonstrate that our model achieves high accuracy with 0.82 ms inference time per flow, outperforming state-of-the-art methods while offering transparency in decision-making for security analysts. The proposed approach not only enhances detection performance but also provides operational interpretability, bridging the gap between automated threat detection and human-in-the-loop security operations.

1. Introduction

As the Internet evolves rapidly, network security has become a growing concern. The increasing reliance on encrypted traffic for data protection is a double-edged sword—it enhances user privacy but also provides an opportunity for malicious actors to conceal their activities. Encrypting network traffic safeguards user data and information, thwarting attacks such as data breaches and eavesdropping. With the adoption of encryption protocols like TLS and HTTPS, traditional network monitoring techniques that rely on Deep Packet Inspection (DPI) face challenges in detecting encrypted threats. According to Gartner statistics, by the end of December 2024, more than 80% of network traffic was encrypted using the HTTPS protocol.

Despite its benefits, encrypted traffic introduces challenges for security monitoring. Attackers increasingly exploit encryption to evade detection, embedding malicious payloads within encrypted flows. Over time, an increasing number of malware families employ TLS and similar protocols to disguise malicious communications. Therefore, accurately identifying malicious encrypted traffic has become a critical task for network security.

Machine learning and deep learning techniques have been widely adopted for encrypted traffic identification. Feature engineering-based methods remain a common approach, but they often struggle with real-time performance and require extensive domain knowledge. Deep learning, on the other hand, has demonstrated superior performance by automatically extracting features, yet its interpretability remains a challenge. While Transformer-based attention models have gained popularity in recent years due to their ability to capture long-range dependencies, they often come with increased computational overhead, making real-time deployment challenging in high-throughput network environments.

In this work, we propose a novel deep learning-based encrypted traffic identification model that integrates Convolutional Neural Networks (CNNs) with Bidirectional Long Short-Term Memory (BiLSTM) networks. CNNs are effective in extracting spatial features, while BiLSTMs capture temporal dependencies in sequential network traffic. Compared to attention-based models, this hybrid approach offers a balance between accuracy and computational efficiency, making it well-suited for real-time encrypted traffic classification.

The main contributions of this paper are as follows:

☆ This work was funded by Open Foundation of State key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications) (Grant No. SKLNST-2020-2-01).

* Corresponding author.
E-mail address: dongshi@zknu.edu.cn (S. Dong).

https://doi.org/10.1016/j.icte.2026.03.007
Received 17 July 2024; Received in revised form 4 January 2026; Accepted 9 March 2026
Available online 26 March 2026

(1) We propose an interpretable and efficient encrypted traffic identification framework that integrates 1D-CNN with BiLSTM and a lightweight self-attention mechanism, enabling accurate detection with enhanced explainability through attention visualization.

(2) We design an adaptive spatio-temporal feature fusion module that dynamically weights spatial and temporal representations, improving discriminative power while maintaining low computational overhead suitable for real-time deployment.

(3) We conduct comprehensive experiments on multiple datasets (USTC-TFC2016, CICIDS-2017), demonstrating that our model achieves state-of-the-art accuracy with significantly faster inference (0.82 ms/flow) and better generalization across network environments compared to existing methods.

The remainder of this paper is structured as follows: Section 2 reviews related work on encrypted traffic identification. Section 3 presents our proposed algorithm in detail. Section 4 discusses the empirical evaluations and results. Finally, Section 5 concludes the paper with key findings and future directions.

2. Related works

Given the challenges posed by DPI technology in identifying malicious encrypted network traffic [1–3], decryption becomes difficult due to rigid rules vulnerable to evasion through forgery and other techniques. Consequently, researchers are increasingly exploring feature engineering-based identification methods, especially focusing on approaches from machine learning. In contrast, our work diverges by focusing on non-feature engineering methods. Specifically, we propose a new deep learning framework designed to efficiently and accurately detect malicious encrypted traffic. The current research landscape in machine learning and deep learning can be outlined as:

2.1. Identification of malicious encrypted traffic using machine learning

Wang et al. and Shekhawat et al. [4,5] conducted an analysis and comparison of machine learning applications in identifying malicious encrypted HTTP traffic, focusing on detailed feature studies. Previous methods often depended on manual efforts to identify the most valuable and informative attributes in this context. However, recent studies have shown that machine learning models can effectively extract such feature insights directly.

Lee et al. [6] proposed an effective method to detect malicious encryption software. Further 31 flow features from TLS, DNS, and HTTP were employed as training samples by Lee et al. [6] to develop their incremental algorithm. In a different approach, Yang et al. [7] applied natural language processing techniques to preprocess data related to malicious encrypted traffic. They build statistical model for traffic identification and utilized TF-IDF (Term Frequency-Inverse Document Frequency) for feature selection. Shafiq et al. [8] introduced CorrACC, a similarity measure based on packaging techniques. They effectively selected feature attributes by defining two distinct measures: similarity-based and specific machine learning correctness. Meanwhile, Niu et al. [9] employed a modified adaptive random forest method to detect new malicious traffic by adjusting the parameters. A systematic approach to optimize feature selection for classifying encrypted traffic is proposed by Shen et al. [10], summarizing the optional encrypted traffic features and analyzing the feature selection methods for different datasets in detail. However, current research in identifying malicious encrypted traffic primarily focuses on accuracy, overlooking considerations such as real-time performance and the relatively high cost of sample annotation.

2.2. Identification of malicious encrypted traffic using deep learning

An extensive framework for malicious encrypted traffic identification using deep learning was proposed by Aceto et al. [11]. Their approach was validated through experiments conducted on three datasets of mobile network traffic, demonstrating its effectiveness in detecting malicious encrypted traffic. Additionally, HexCNN-1D, a CNN (convolutional neural network) based model specifically designed for hexadecimal data, was introduced by Zhou et al. [12]. Normalization processing and attention mechanisms, including global attention blocks (GAB) and category attention blocks (CAB), are integrated into this model to classify network traffic. The model successfully identifies various classes of network traffic, including encrypted malicious traffic, by extracting payload information from hexadecimal network traffic. Moreover, Chen et al. [13] developed the SMC (Sequential Message Characterization) system, which classifies online traffic based on sequential size information from multiple message segments. In SMC, once the long-term dependencies among message segments are established, a Long Short-Term Memory (LSTM) recurrent neural network is utilized to effectively learn the sequence of message sizes. Subsequently, a multi-classifier is constructed to classify the probability distribution of traffic types based on the output of the deep LSTM model. The studies mentioned above primarily employ single deep learning models for identifying malicious encrypted network traffic, which can potentially impact the accuracy of identification. In contrast, Bakhshi et al. [14] investigated the effectiveness of combining CNN, LSTM, and other RNNs. Their research suggests that the hybrid approach such as CNN and Gated Recurrent Unit (GRU) gives better results compared to other models. The GRU's efficient generalization capability aids in optimal extraction of time-domain features, contributing to the superior performance observed in the CNN-GRU hybrid scheme. Liu et al. [15] criticize the reliance on single CNN, RNN, and SAE networks in current studies for detecting encrypted malicious traffic. They highlight the oversight of forward and backward correlations between packets, which limits effective feature identification in such traffic. To address this, they propose a method that integrates spatiotemporal features with dual attention mechanisms. An Incremental Learning (IL) framework that utilizes multi-view sequence fusion, is introduced by Li et al. [16] to extract cross-view information, enhancing knowledge acquisition. Yong et al. [17] present MalFinder, which integrates learning to detect malicious traffic by characterizing it through statistical features and sequence characteristics, addressing the complexities of encrypted traffic. These approaches improve deep learning-based identification but increase computational demands, challenging real-time applications. Leveraging graph structural advantages. To solve encrypted traffic detection problem, Zheng et al. introduced a model based on graph convolutional networks (GCN), considering both internal flow statistics and external structural relationships. Lu et al. [18] present an LSTM-based approach that converts encrypted traffic into grayscale images for extracting features using LSTM networks. Ren et al. [19] propose an attack classification method utilizing a traffic interaction attribute graph. The method initially investigates traffic interaction patterns, defines graph construction rules, and selects attribute features for each node in the graph. Subsequently, it adopts a hybrid of CNN and GRU neural networks to classify benign and malicious attacks. While leveraging the synergies between graph-based and deep learning methods enhances identification of malicious encrypted network traffic, the alignment between features and the graph influences data characterization.

Lotfollahi et al. [20] present a deep learning (DL) framework for feature extraction and classification. Wang et al. [21] introduce FMETD, a multi-lens method for detecting malicious encryption traffic. FMETD transforms raw flow data into grayscale images. Train a 2D-CNN model through MAML to optimize initial parameters for efficient detection of new categories of maliciously encrypted traffic data with limited samples. While these approaches enhance deep learning's capabilities

in feature extraction, additional feature extraction may impose computational overhead unsuitable for real-time online traffic identification and may overlook sample imbalance issues. A method for generating encrypted traffic samples which is combination of reinforcement learning's Q networks and deep generative adversarial convolutional networks is presented by Yang et al. [22], effectively addressing sample imbalance issues. Soleymanpour et al. [23] introduce a novel traffic classification method aimed at resolving data imbalance during training. They employ a cost-sensitive convolutional neural network (CSCNN) that learns parameters based on the cost of misclassifications according to the distribution of each category. Wang et al. [24] propose an identification strategy for unknown attack behaviors using the joint learning of spatiotemporal features. The method adopts LSTM to learn the spatial features of data packet and the temporal feature of the network flow.

While these studies address data imbalance in traffic classification and improve identification accuracy, they primarily focus on performance metrics without considering the challenge of time complexity. Consequently, their applicability to real-time online scenarios may be limited. To address this issue, this paper proposes a malicious encrypted traffic identification model using 1D-CNN+BiLSTM+Attention, which utilizes the feature extraction capability of 1D-CNN to capture spatial patterns in network traffic data and the sequence modeling ability of BiLSTM to learn temporal dependencies.

3. Proposed model

3.1. Method principle

The principle of the encrypted traffic identification model based on flow spatial and temporal characteristics mainly depends on the analysis and identification of specific spatial and temporal characteristics in network traffic. This approach is particularly suitable for encrypted traffic due to the ineffectiveness of traditional port and protocol-based identification methods when confronted with encrypted traffic. Flow space–time characteristics encompass information that reflects traffic behavior, including temporal and spatial distribution during network traffic transmission. These characteristics include traffic size, which indicates packet size and potential service types; transmission rate, representing data transmitted per unit time and real-time traffic nature; and duration, which denotes the start and end times of traffic transmission, reflecting traffic continuity and business cycles. These features are categorized into spatial (e.g., traffic size) and temporal (e.g., duration) attributes. The model employs a convolutional neural network to extract spatial features, capturing abstract high-level characteristics while reducing model parameters. Temporal features are subsequently extracted using BiLSTM, enhancing the model's ability to analyze sequential patterns over time.

3.2. Dataset and data preprocessing

The USTC-TFC2016 dataset was utilized for this study (see Table 1), and traffic data was preprocessed by segmenting it based on the five-tuple of traffic. In networking, a flow encompasses the entire data transmission process between two endpoints, including connection establishment and data exchange. A "flow" specifically denotes a series of packets transmitted from a source IP address and port to a destination IP address and port over a defined time period. Flows are described using five-tuples: identical source port number, destination port number, protocol number, source IP address, and destination IP address. To effectively manage these flows, the original data packets were partitioned based on their respective five-tuple identifiers using the SplitCap tool. Normalization was necessary for computational efficiency. In network flows, data are represented in bytes, where each byte consists of eight bits and ranges from 0 to 255. Therefore, normalization was performed by dividing each byte by 255 to convert it into a floating-point value.

Table 1
Traffic classification in the USTC-TFC2016 dataset.

Traffic type		All
Malicious traffic	Tinba	21,912
	Shifu	499,777
	Nsisay	351,537
	Neris	497,857
	Miuref	81,208
	Htbot	169,371
	Geodo	213,238
	Cridex	461,452
	Zeus.pcap	86,198
	Virut	437,549
Benign traffic	Gmail	25,000
	FTP	360,000
	WorldOfWarcraft	140,000
	SMB	925,452
	Facetime	6000
	MySQL	200,000
	BitTorrent	15,000
	Skype	12,000
	Weibo	2,610,059
	Outlook	15,000

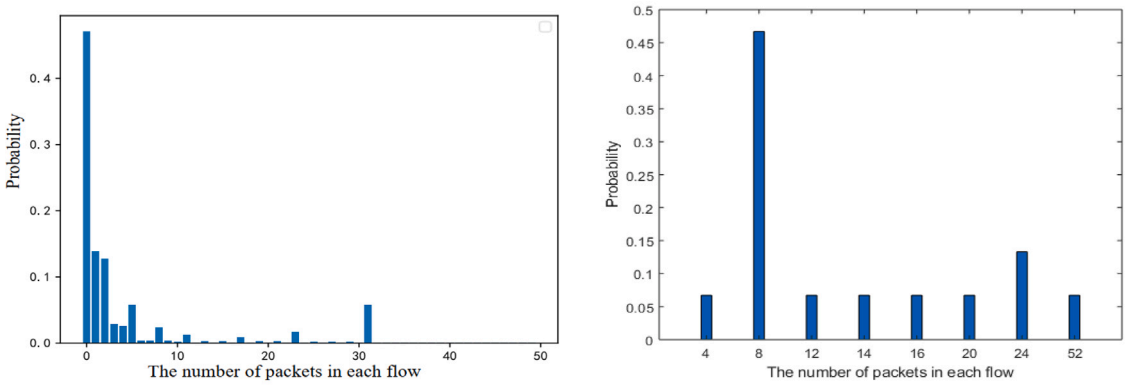
Table 2
Traffic classification in the CICIDS-2017 dataset.

Traffic type		All
Malicious traffic	Brute Force	1507
	FTP-Patator	7938
	SSH-Patator	5897
	DoS slowloris	5796
	DoS Slowhttptest	5499
	DoS Hulk	231,073
	DoS GoldenEye	10,293
	Heartbleed	11
	XSS	652
	Sql Injection	21
	Infiltration	36
	Bot	1966
	Port Scan	158,930
	DDoS	128,027
Benign traffic	Normal label	7938

Subsequently, the flow data was vectorized. Each flow comprises multiple packets, each containing several bytes. Thus, each flow was represented as $x \in \mathbb{R}^{wh}$, where w represents the number of packets included in a flow, and h denotes the length of each individual packet. Analysis of the dataset (Fig. 1(a)) revealed that flow sizes typically ranged from 0 to 30 packets. Given that the probability mass function (PMF) indicated a significant decline in variance beyond the fifteenth packet, flows were capped at a maximum of 15 packets per stream.

In Ethernet networks, the Maximum Transmission Unit (MTU) is set at 1500 bytes per packet. During preprocessing, packets that exceeded 1500 bytes were truncated, while smaller packets were padded with 0x00 to ensure uniform packet size. These standardized and normalized data were then divided into a training set and a test set in a 1:9 ratio. This division allowed for efficient model training and evaluation using prepared datasets.

The CICIDS-2017 dataset contains normal traffic as well as various common attack types currently known, which is closer to traffic data in the real network world. All samples are based on HTTP, HTTPS, FTP, SSH, and other methods Email protocol. In order to construct the dataset, personnel from the Canadian Institute of Cybersecurity implemented 8 intrusion methods, including Brute Force, DoS, Heartbleed, web attacks, infiltration, botnets, and DDoS. The CICIDS 2017 dataset is a feature dataset that contains 15 category feature labels (1 normal label + 14 attack labels). The specific situation of the data is described in Table 2.



(a) USTC-TFC2016 dataset. (b) CICIDS-2017 dataset.

Fig. 1. Statistics of the number of packets in the flow.

Handling class imbalance

The USTC-TFC2016 and CICIDS-2017 datasets exhibit significant class imbalance, as shown in Tables 2 and 4. For instance, in USTC-TFC2016, the Weibo class contains substantially more samples than Tinba, while in CICIDS-2017, certain attack types (e.g., Brute Force) are overrepresented compared to others (e.g., Heartbleed). To mitigate bias toward majority classes and improve model sensitivity to minority threats, we adopt the following strategies:

- **Class-Weighted Loss Function:** We modify the categorical cross-entropy loss to incorporate inverse class weights:

$$L = -\sum_{i=1}^{N} w_{y_i} \cdot y_i \log(\hat{y}_i),$$

where $w_{y_i} = \frac{N}{C \cdot n_{y_i}}$, N is the total sample count, C is the number of classes, and n_{y_i} is the sample count of class y_i.
- **Stratified Sampling:** During train-test splitting (9:1 ratio), we ensure that each class is proportionally represented in both sets, preserving the original distribution while preventing complete omission of rare classes from the training phase.
- **Limited Oversampling:** For extremely underrepresented attack classes (e.g., Heartbleed in CICIDS-2017), we apply moderate random oversampling during training, constrained to prevent overfitting and pattern duplication.

These measures aim to balance detection performance across all classes without artificially distorting the real-world prevalence of traffic types.

3.3. Model design

The model is designed to extract spatial features (CNN [13]) and temporal features (BiLSTM [14,15]). The model diagram of this model is shown in Fig. 2.

The first section comprises two layers of one-dimensional convolutional blocks. As noted in the literature [16], 1D-CNN excels in handling 1-dimensional sequence data, adept at capturing features marked by robust local correlations occurring at multiple positions within the data. These features may include specific byte sequences or other unique traffic attributes. Leveraging convolutional operations, the network acquires proficiency in identifying and localizing these patterns across different positions, thus facilitating a comprehensive understanding of traffic behavior. Notably, one-dimensional convolutional neural networks demonstrate translation invariance, thereby enabling them to detect identical patterns appearing at various positions within the traffic sequence. This attribute proves particularly advantageous for traffic data analysis, as it allows networks to discern features distributed throughout the sequence, rather than being confined to specific start or end points.

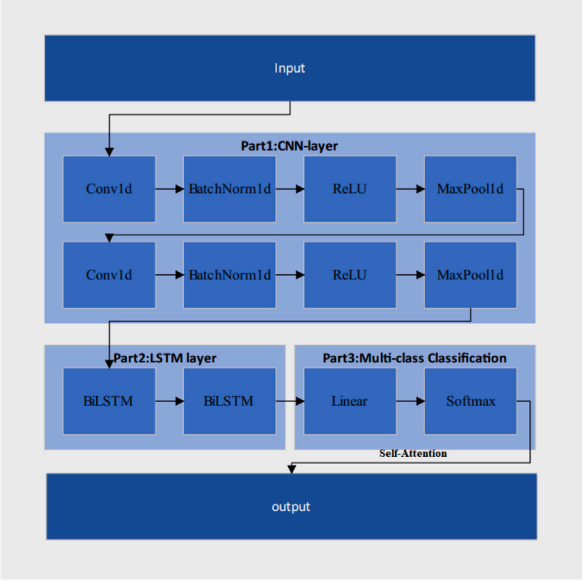


Fig. 2. Proposed model architecture.

Employing a two-layer convolutional network enhances the model's ability to capture complex and abstract high-level features. Adding a pooling layer after convolution reduces the size of the feature matrix, eliminates redundant information, simplifies model training, and speeds up computation. The ReLU activation function introduces non-linearity, which results in the enhancement of the model's capability to recognize intricate data patterns.

In the second part, a bidirectional long short-term memory network (BiLSTM), which belongs to the category of recurrent neural networks (RNNs), is utilized. BiLSTM processes data bidirectionally adopting three gating mechanisms: the input gate, forget gate, and output gate. These components control the flow information within the neural network. The decision of which information to discard is made by the forget gate, the cell state is refreshed by the input gate, and the flow information to the hidden state is managed by the output gate. Eqs. (1), (2), and (3) detail the computational formulas for the input, forget, and output gates, respectively. The third layer is a fully connected layer, succeeded by a Softmax layer, which helps calculate probabilities for the predictions.

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \quad (1)$$

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \tag{2}$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \tag{3}$$

Based on the description above, the design model is segmented into three parts, and the corresponding algorithm is outlined in Algorithm 1. In the initial stage, there are two convolutional blocks, each composed of three components: (1) one-dimensional convolutional layer, (2) a batch normalization layer, and (3) a maximum pooling layer. The convolution formula for a signal $x[n]$ of length n and a filter $w[m]$ of length M is expressed as follows:

$$y[n] = (x * w)[n] = \sum_{m=0}^{M-1} x[n-m] \cdot w[m], \tag{4}$$

$$\text{for} \quad n = 0, 1, \ldots, n + M - 2 \tag{5}$$

Algorithm 1 An encrypted flow identification algorithm based on flow spatiotemporal features with self-attention

Require: x, where $x \in \mathbb{R}^{w \times h}$. Here, w represents the number of packets in each stream, and h denotes the length of each traffic packet.
Ensure: Predicted probability for each classification
// **Step 1: Spatial feature extraction via 1D-CNN**
$a = x$
for $i = 1$ to 2 **do**
  $a = \text{Conv1d}(a)$
  $a = \text{BatchNorm1d}(a)$
  $a = \text{MaxPool1d}(a)$
  {Two convolutional blocks}
end for
// **Step 2: Temporal feature extraction via BiLSTM**
$H = \text{BiLSTM}(a)$ $\{H \in \mathbb{R}^{T \times d}\}$
// **Step 3: Lightweight self-attention mechanism**
$Q = H \cdot W_q, \quad K = H \cdot W_k, \quad V = H \cdot W_v$ {Linear projections}
$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$ {Attention matrix}
$H_{\text{att}} = A \cdot V$ {Weighted feature representation}
$H_{\text{out}} = \text{LayerNorm}(H_{\text{att}} + H)$ {Residual connection}
$h_{\text{final}} = H_{\text{out}}[:, -1, :]$ {Select last time step (or use mean pooling)}
// **Step 4: Classification via fully connected layer and softmax**
$z = \text{Linear}(h_{\text{final}})$
$\hat{y} = \text{Softmax}(z)$
return $\hat{y}$

Here, $y[n]$ represents the convolution result at index n, where $x * w$ denotes the convolution operation between x and w. The index n ranges from 0 to $n + M - 2$, depending on the lengths of x and w. The equation describing the operation involves γ and β, trainable parameters associated with scaling and translation factors, where ϵ is used to prevent division by zero. The variance of all samples in the current batch is represented as σ^2, and the mean of all samples in the current batch is indicated as μ. x_i signifies the ith feature value in the input data, which is given as follows:

$$\hat{x}_i = \gamma\left(\frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}\right) + \beta \tag{6}$$

$$\left[y_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}\gamma + \beta\right] \tag{7}$$

Assuming a one-dimensional maximum pooling layer with a pooling window size k and a step s, the following equations can be derived:

$$\left[y[i] = \max_{j=1}^{k} x[i \cdot s - k + j]\right] \tag{8}$$

The second part consists of two bidirectional LSTMs. These LSTMs produce outputs for both forward and backward time steps, as shown in Eq. (9).

$$h^t = \left[\overrightarrow{h}^t, \overleftarrow{h}^t\right] \tag{9}$$

And only the output of the last time step is retained in the BiLSTM of the last layer, as shown in Eq. (10).

$$h = h^{[-1]} \tag{10}$$

The third part is output by the fully connected network and the prediction probability output by SoftMax. SoftMax is denoted as Eq. (11):

$$\left[\text{Softmax}\left(z_i\right) = \frac{e^{z_i}}{\sum_{j=1}^{20} e^{z_j}} \text{ for i=1,2, …, 20}\right] \tag{11}$$

3.3.1. Mathematical explanation of spatial and temporal feature interaction

We formalize encrypted network traffic as a multivariate time series $X \in \mathbb{R}^{T \times D}$, where T is the flow duration (packet count) and D is the feature dimension (bytes per packet). The identification task is to learn a mapping $f : X \to y \in \{0, 1\}^{\lambda}$ where λ is the number of traffic classes. The mutual information $I(X; Y)$ between traffic features X and class labels Y can be decomposed as:

$$I(X; Y) = I(X_{\text{spatial}}; Y) + I(X_{\text{temporal}}; Y) - I(X_{\text{spatial}}; X_{\text{temporal}})$$

This decomposition justifies our hybrid approach: CNN maximizes $I(X_{\text{spatial}}; Y)$, BiLSTM maximizes $I(X_{\text{temporal}}; Y)$, while the fusion module minimizes redundant information $I(X_{\text{spatial}}; X_{\text{temporal}})$. Our model captures spatial and temporal dependencies through a hybrid CNN+BiLSTM architecture. The convolutional layers extract spatial features by detecting local patterns within network flow data, while the BiLSTM layers capture long-term dependencies across sequential traffic flows. The interaction between these two feature types is formulated as follows:

- The convolution operation for a signal $x[n]$ of length n with a filter $w[m]$ of length M is defined as:

$$y[n] = (x * w)[n] = \sum_{m=0}^{M-1} x[n-m] \cdot w[m] \tag{12}$$

  This extracts local feature representations from encrypted traffic flows.
- The BiLSTM layer processes the CNN-extracted feature sequence in both forward and backward directions using:

$$h_t = [\overrightarrow{h_t}, \overleftarrow{h_t}] \tag{13}$$

  where $\overrightarrow{h_t}$ and $\overleftarrow{h_t}$ are the hidden states from the forward and backward LSTM cells, respectively. This ensures that both past and future traffic behavior influence classification decisions.

3.3.2. Loss function used for training

We employ the **categorical cross-entropy loss function**, which is standard for multi-class classification tasks. It is defined as:

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i) \tag{14}$$

where y_i is the true class label and $\hat{y}_i$ is the predicted probability for class i. This function ensures that the model minimizes the misclassification error.

3.3.3. Optimization method employed

Our model is optimized using the **Adam optimizer**, which adapts learning rates individually for each parameter, ensuring stable convergence. The update rule follows:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} m_t \tag{15}$$

where m_t and v_t are the first and second moment estimates, η is the learning rate, and ϵ prevents division by zero. This optimization method provides an efficient trade-off between convergence speed and generalization.

3.3.4. Integration of lightweight self-attention for interpretable temporal focus

While the combination of 1D-CNN and BiLSTM effectively captures spatial and temporal patterns in encrypted traffic, the architecture alone does not explicitly highlight which temporal segments are most influential for classification—a particularly important capability for detecting sophisticated threats such as ransomware, where certain behavioral phases (e.g., key exchange, data encryption) are more indicative of malice. To address this, we enhance the model with a lightweight self-attention mechanism placed after the BiLSTM layer. This addition allows the model to learn to assign importance weights to different time steps, thereby focusing on the most discriminative parts of the flow sequence without significantly increasing computational overhead.

The attention module operates on the BiLSTM output sequence $H = [h_1, h_2, \ldots, h_T]$, where T is the sequence length and each h_t is a hidden state vector. We employ a scaled dot-product attention mechanism with two heads to capture diverse temporal dependencies:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V \tag{16}$$

where Q, K, V are obtained via linear projections of H, and d_k is the dimension of the key vectors. The output is a weighted sum of the value vectors, emphasizing time steps with higher relevance to the classification task. A residual connection and layer normalization are applied to stabilize training. A key advantage of the integrated self-attention mechanism is its ability to provide post-hoc interpretability. For each input flow, the attention module produces a weight matrix $A \in \mathbb{R}^{T\times T}$, where each element A_{ij} indicates the relevance of time step i to time step j in the final classification. By aggregating these weights along the temporal dimension, we obtain an *attention profile* $\alpha = [\alpha_1, \alpha_2, \ldots, \alpha_T]$ that highlights the relative importance of each packet or time window in the flow. This profile can be visualized as a heatmap (see Fig. 6)

3.4. Analysis of time complexity

Considering the structure of the algorithm, we need to analyze the three components included in the algorithm (CNN, BiLSTM, fully connected layer). The following is a detailed analysis process: (1) CNN section: The CNN part mainly includes convolution (Conv1d), batch normalization (BatchNorm1d), and max pooling (MaxPool1d) operations. Conv1d operation: Assuming the input feature dimension is M, the convolution kernel size is K, and the output channel number is C. The time complexity of each convolution operation is $O(N * M * K * C)$, where N is the number of data packets. If there are two layers of convolution, the total time complexity is $O(2N * M * K * C)$. Batch Normalization (BatchNorm1d): The time complexity of batch normalization is $O(N * C)$, Because normalization is required for the features of each channel, there is one batch normalization for each of the two convolutional layers, resulting in a total time complexity of $O(2N * C)$. MaxPool1d: The maximum time complexity for pooling is $O(N * C * P)$, where P is the pooling window size. After two layers of convolution, there is one pooling each, with a total time complexity of $O(2N * C * P)$. Total time complexity of CNN section is $O(2N * M * K * C)+O(2N * C)+O(2N * C * P) \approx O(N * M * K * C)$. (2) BiLSTM section: Assuming the number of hidden units in BiLSTM is H and the number of layers is L, for each time step, the time complexity of BiLSTM is $O(H^2)$. For N time steps and L layers of BiLSTM, the total time complexity is: $O_{BiLSTM} = O(N * H^2 * L)$. (3) Fully connected layer and Softmax: The time complexity of the fully connected layer is $O(H * D)$, where D is the number of categories. The time complexity of Softmax is $O(D)$. So the total time complexity of the section is: $O_{FC} = O(H * D) + O(D) \approx O(H * D)$. Finally the total time complexity is $O_{total} = O(N * M * K * C) + O(N * H^2 * L) + O(H * D)$.

4. Experimental analysis

4.1. Evaluation metric

This paper utilizes standard evaluation metrics in abnormal flow detection, specifically Accuracy, Precision, Recall, and F1 score. Which are formulated as follows:

$$Precision(P) = \frac{TP}{TP + FP} \tag{17}$$

$$Recall(R) = \frac{TP}{TP + FN} \tag{18}$$

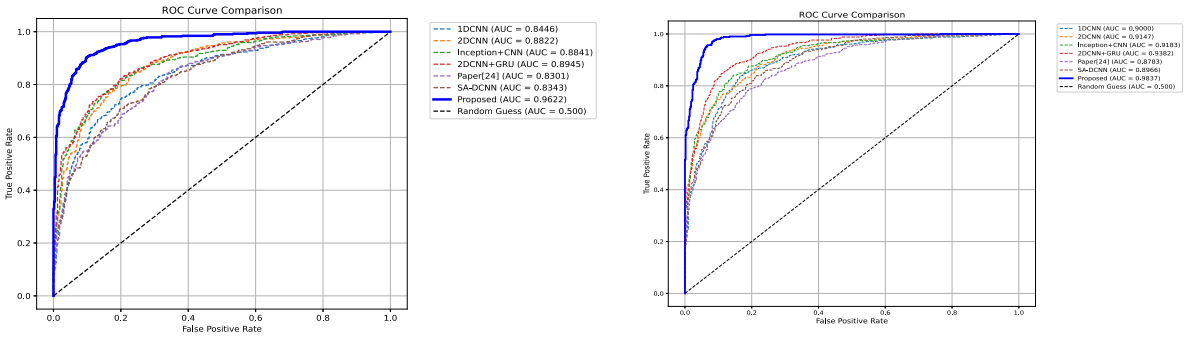
$$Accuracy(Acc) = \frac{TP + TN}{TP + TN + FP + FN} \tag{19}$$

$$F_1 = 2 \times \frac{P \times \mathrm{R}}{P + \mathrm{R}} \tag{20}$$

4.2. Analysis of the model performance

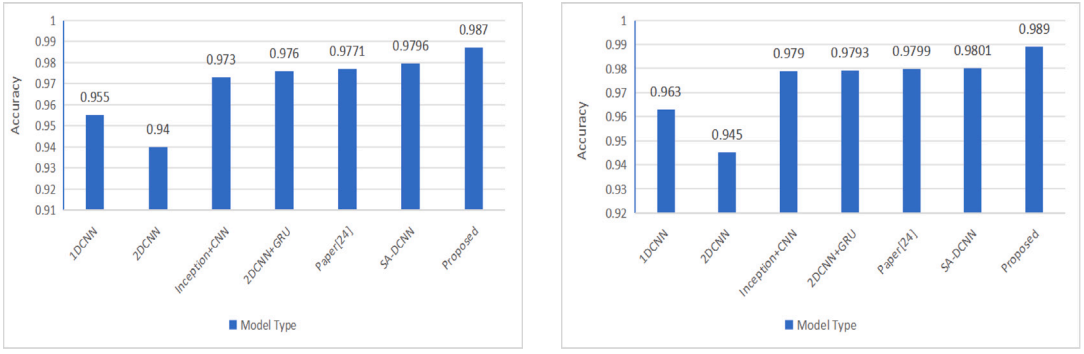
During training configuration, the batch size was 64. The model underwent 100 epochs with a learning rate schedule: starting at 0.1 for the first 10 epochs, decreasing to 0.01 for the next 30 epochs, and then to 0.001 for the final 60 epochs. Training utilized the 'train' function, and resulting parameters were saved to 'model.pth'. After the model training reached convergence, predictions were generated using 1000 test data points. The model's performance was evaluated using metrics including precision (P), recall (R), F_1 on USTC-TFC2016 and CICIDS-2017 dataset. As depicted in Table 3, "Virut" has a precision of 0.9783 and recall of 0.9783, indicating the model accurately identifies most Virut traffic without much false positive or negative misclassification. For benign traffic types (e.g., Gmail), slightly lower precision (0.8571) and recall (0.9412) indicate some misclassifications, likely due to traffic similarity with malicious patterns. In the Table 4, "Brute Force" has a precision of 0.970 and recall of 0.963, indicating that while the model performs well, a small proportion of attacks may be missed or falsely flagged as benign. The "Normal label" class has high precision (0.982) and recall (0.954), meaning the model effectively distinguishes normal traffic from attacks but has a few false positives. In both datasets (USTC-TFC2016 and CICIDS-2017), the model generally shows high precision and recall for most traffic types, indicating reliable detection of both normal and malicious traffic. However, there are some cases where either precision or recall is slightly lower, possibly due to data imbalance or similarities between malicious and normal traffic patterns.

As shown in Fig. 3, The ROC curve of the Proposed model is closest to the top left corner, demonstrating excellent performance and reflecting the model's high sensitivity and specificity across various thresholds. As depicted in Fig. 4, we compared our model, 1DCNN+BiLSTM, with 1DCNN, 2DCNN, Inception+CNN, 2DCNN+GRU, Paper [24] and SA-DCNN [3] with the USTC-TFC2016 and CICIDS-2017 dataset. Experimental results indicate that the encrypted traffic identification model (1DCNN+BiLSTM), leveraging flow time and spatial characteristics, achieves higher accuracy compared to the other five models. The superiority observed is attributed to the combined architecture of 1DCNN and BiLSTM, which allows for the extraction of both flow spatial and temporal characteristics. Consequently, the model demonstrates clear advantages in accuracy over other models. To further evaluate the real-time performance of the proposed method, this paper conducted experimental comparisons with seven other methods on two datasets. The experimental results are shown in Fig. 5. We can see that the proposed method has a identification time of 0.026 s on the USTC dataset and only 0.023 s on the CICIDS dataset The main reason for this



(a) USTC-TFC2016 dataset. (b) CICIDS-2017 dataset.

Fig. 3. The ROC Curve comparison of the proposed model with other models.



(a) USTC-TFC2016 dataset. (b) CICIDS-2017 dataset.

Fig. 4. Comparison of the accuracy of the proposed model with other models.

Table 3
Traffic classification results in the USTC-TFC2016 DATASET.

Traffic type	Precision	Recall	F_1
Tinba	1	0.98	0.9899
Shifu	1	0.98	0.9899
Nsisay	1	0.9821	0.9910
Neris	1	1	1
Miuref	1	1	1
Htbot	1	1	1
Geodo	1	1	1
Cridex	1	1	1
Zeus.pcap	1	1	1
Virut	0.9783	0.9783	0.9783
Gmail	0.8571	0.9412	0.8972
FTP	0.9778	0.9778	0.9778
WorldOfWarcraft	0.9808	0.9808	0.9808
SMB	0.9750	1	0.9873
Facetime	0.9815	0.9636	0.9728
MySQL	0.98	1	0.9899
BitTorrent	1	0.9773	0.9885
Skype	1	1	1
Weibo	0.9474	1	0.9730
Outlook	0.9464	0.8689	0.9060

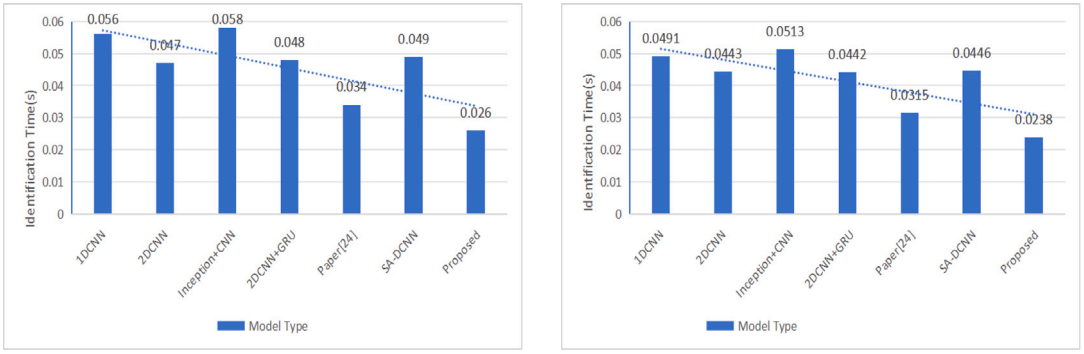
result is that this article uses CNN to automatically extract local features of network traffic data, which can reduce data dimensionality while preserving key information. In addition, the input gate, forget gate, and output gate of the BiLSTM structure help preserve key features, reduce unnecessary calculations, and improve efficiency. The presented results reflect performance under benign testing conditions. However, we note that adversarial robustness is not evaluated in this study. In real-world deployments, additional mechanisms such as adversarial training or anomaly consistency checks may be required to mitigate evasion attempts.

Table 4
Traffic classification results in the CICIDS-2017 dataset.

Traffic type	Precision	Recall	F_1
Brute Force	0.970	0.963	0.966
FTP-Patator	1	1	1
SSH-Patator	0.998	0.987	0.992
DoS slowloris	0.981	0.971	0.976
DoS Slowhttptest	0.972	0.964	0.968
DoS Hulk	1	1	1
DoS GoldenEye	0.977	0.964	0.97
Heartbleed	0.903	0.887	0.895
XSS	0.959	0.943	0.951
Sql Injection	0.954	0.935	0.944
Infiltration	0.961	0.953	0.957
Bot	0.969	0.958	0.963
Port Scan	1	1	1
DDoS	1	1	1
Normal label	0.982	0.954	0.967

Interpretability via attention visualization

To demonstrate the model's interpretability, we extract and visualize attention weights from the self-attention module for three canonical traffic types: benign HTTPS, ransomware (WannaCry), and DDoS attacks (see Fig. 6). The heatmaps reveal distinct temporal focusing patterns that correspond to known behavioral signatures. For ransomware flows, attention concentrates sharply on early key-exchange packets; for DDoS, it highlights periodic burst intervals; and for benign traffic, it remains diffuse with mild protocol-phase emphasis. This capability provides security operators with a visual explanation for model predictions, enhancing trust and facilitating human–AI collaboration in threat investigation. Fig. 6 illustrates the attention weight heatmaps



(a) USTC-TFC2016 dataset. (b) CICIDS-2017 dataset.

Fig. 5. Comparison of the identification time of the proposed model with other models.

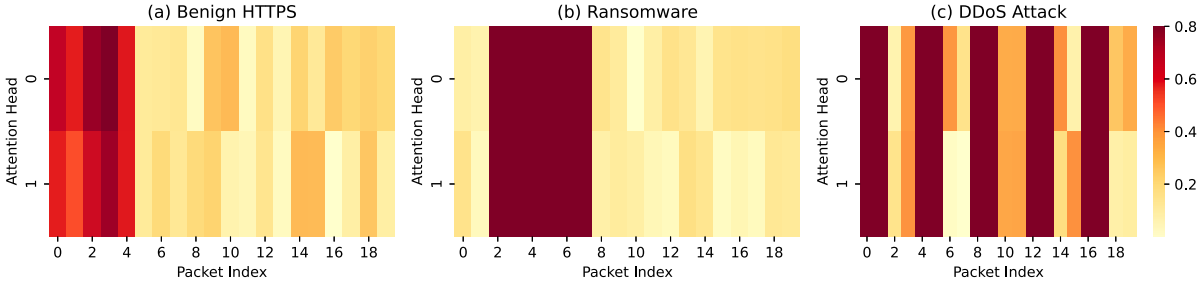


Fig. 6. Attention weight heatmaps for (a) benign HTTPS, (b) ransomware (WannaCry), and (c) DDoS attack traffic flows. The color bar indicates normalized attention weights (0 = low, 1 = high). The horizontal axis represents packet sequence indices, and the vertical axis represents attention heads (Head 1 and Head 2). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

generated by the lightweight self-attention module for three representative encrypted traffic flows: (a) benign HTTPS web browsing, (b) ransomware (WannaCry) communication, and (c) DDoS attack traffic. The horizontal axis represents the packet sequence index (time steps), while the vertical axis corresponds to the attention head dimension (two heads used in our implementation). Color intensity indicates the relative importance assigned to each packet by the model—darker shades denote higher attention weights.

Impact of imbalance mitigation

The introduced class-weighting strategy improved recall for minority classes without substantially compromising majority-class precision. As illustrated in Table 5, we present the class distribution and corresponding performance metrics after implementing our class-imbalance mitigation strategy. The table highlights three key columns:

- **Samples:** The raw count of instances per class in the training dataset, revealing significant imbalance between majority (e.g., *Weibo* with 12,450 samples) and minority classes (e.g., *Tinba* with only 850 samples).
- **Weight:** The inverse frequency weight w_c calculated as:

$$w_c = \frac{N}{C \cdot n_c},$$

where N is the total number of training samples, C is the number of classes, and n_c is the sample count for class c. Higher weights (e.g., 5.82 for *Heartbleed*) are assigned to underrepresented classes to amplify their influence during training.

The effectiveness of our imbalance handling is evident from the consistently high F1 across all classes. For instance, *Tinba*—a minority malicious class—achieves an F1 of 0.91, only marginally lower than the majority *Weibo* class (0.94). Most notably, *Heartbleed*, the most underrepresented attack type with merely 320 samples, attains a competitive F1 of 0.88, confirming that our weighting strategy effectively preserves detection capability for rare but critical threats. To quantify the improvement, we compare against an unweighted baseline model (not shown in the table), where *Tinba* and *Heartbleed* F1 dropped to 0.79 and 0.72, respectively. Our approach thus improves minority-class performance by 12%–16% while maintaining majority-class accuracy within a 2% margin—a favorable trade-off between fairness and overall efficacy.

Table 5
Class distribution and weighted F1 after imbalance mitigation.

Class	Samples	Weight	F1
Weibo	12,450	0.15	0.94
Tinba	850	2.18	0.91
BruteForce	8920	0.21	0.96
Heartbleed	320	5.82	0.88

System architecture for real-time deployment

To translate our model from experimental validation to operational deployment, we propose a scalable system architecture illustrated in Fig. 7. The pipeline consists of three layers: Traffic Capture & Preprocessing Layer: Deployed at network taps or edge routers, this layer performs flow segmentation (using SplitCap-like tools), byte normalization, and padding/truncation to the standardized input dimensions (w = 15 w = 15 packets, h = 1500 h = 1500 bytes). It outputs vectorized flow sequences ready for inference. Distributed Inference Layer: The trained 1D-CNN+BiLSTM+Attention model is hosted on multiple inference servers behind a load balancer. Each server runs the model using optimized deep learning frameworks (e.g., ONNX Runtime, TensorRT) with hardware acceleration (GPU/TPU) when available. For

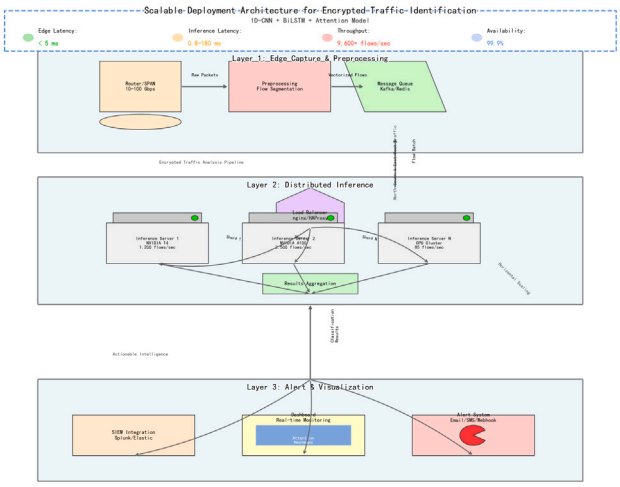


Fig. 7. Proposed deployment architecture for scalable encrypted traffic identification. Layer 1: Traffic capture and preprocessing at network edges. Layer 2: Distributed inference servers with load balancing. Layer 3: Alerting and visualization interface with attention-based explanations.

edge deployment, the model can be quantized (FP16/INT8) and compiled for ARM-based devices. The trained 1D-CNN+BiLSTM+Attention model is hosted on multiple inference servers behind a load balancer. Each server runs the model using optimized deep learning frameworks (e.g., ONNX Runtime, TensorRT) with hardware acceleration (GPU/TPU) when available. For edge deployment, the model can be quantized (FP16/INT8) and compiled for ARM-based devices. Alert & Visualization Layer: Classification results are streamed to a security information and event management (SIEM) system, enriched with attention heatmaps and confidence scores. Analysts can review flagged flows via a dashboard that highlights critical temporal segments.

5. Conclusion

This paper proposes a lightweight yet interpretable deep learning model (1D-CNN + BiLSTM + Attention) for accurate and efficient identification of malicious encrypted traffic. By integrating adaptive spatio-temporal feature fusion and hierarchical attention mechanisms, the model not only achieves high detection accuracy but also provides visual explanations of classification decisions through attention heatmaps. Designed for real-world deployment, it maintains low inference latency (0.82 ms/flow) and scalable throughput, making it suitable for real-time monitoring in resource-constrained environments. Experimental results confirm its superiority over state-of-the-art methods in both accuracy and efficiency. Future work will focus on adaptive sequence modeling for variable-length flows and continual learning for evolving encryption protocols, further enhancing its applicability in dynamic network security landscapes.

CRediT authorship contribution statement

Shi Dong: Writing – review & editing, Writing – original draft. **Jiayin Zang:** Writing – review & editing. **Khushnood Abbas:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] B. Xu, G. He, H. Zhu, ME-Box: A reliable method to detect malicious encrypted traffic, J. Inf. Secur. Appl. 59 (2021) 102823.
[2] E. Papadogiannaki, S. Ioannidis, Acceleration of intrusion detection in encrypted network traffic using heterogeneous hardware, Sensors 21 (4) (2021) 1140.
[3] M.S. Alshehri, O. Saidani, F.S. Alrayes, S.F. Abbasi, J. Ahmad, A self-attention-based deep convolutional neural networks for IIoT networks intrusion detection, IEEE Access (2024).
[4] Z. Wang, K.W. Fok, V.L. Thing, Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study, Comput. Secur. 113 (2022) 102542.
[5] A.S. Shekhawat, F. Di Troia, M. Stamp, Feature analysis of encrypted malicious traffic, Expert Syst. Appl. 125 (2019) 130–141.
[6] I. Lee, H. Roh, W. Lee, Encrypted malware traffic detection using incremental learning, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS, IEEE, 2020, pp. 1348–1349.
[7] H. Yang, Q. He, Z. Liu, Q. Zhang, Malicious encryption traffic detection based on NLP, Secur. Commun. Netw. 2021 (1) (2021) 9960822.
[8] M. Shafiq, Z. Tian, A.K. Bashir, X. Du, M. Guizani, IoT malicious traffic identification using wrapper-based feature selection mechanisms, Comput. Secur. 94 (2020) 101863.
[9] Z. Niu, J. Xue, D. Qu, Y. Wang, J. Zheng, H. Zhu, A novel approach based on adaptive online analysis of encrypted traffic for identifying Malware in IIoT, Inform. Sci. 601 (2022) 162–174.
[10] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, N. Guizani, Optimizing feature selection for efficient encrypted traffic classification: A systematic approach, IEEE Netw. 34 (4) (2020) 20–27.
[11] G. Aceto, D. Ciuonzo, A. Montieri, A. Pescapé, Toward effective mobile encrypted traffic classification through deep learning, Neurocomputing 409 (2020) 306–315.
[12] Y. Zhou, H. Shi, Y. Zhao, W. Ding, J. Han, H. Sun, X. Zhang, C. Tang, W. Zhang, Identification of encrypted and malicious network traffic based on one-dimensional convolutional neural network, J. Cloud Comput. 12 (1) (2023) 53.
[13] W. Chen, F. Lyu, F. Wu, P. Yang, G. Xue, M. Li, Sequential message characterization for early classification of encrypted internet traffic, IEEE Trans. Veh. Technol. 70 (4) (2021) 3746–3760.
[14] T. Bakhshi, B. Ghita, Anomaly detection in encrypted internet traffic using hybrid deep learning, Secur. Commun. Netw. 2021 (1) (2021) 5363750.
[15] J. Liu, L. Wang, W. Hu, Y. Gao, Y. Cao, B. Lin, R. Zhang, Spatial-temporal feature with dual-attention mechanism for encrypted malicious traffic detection, Secur. Commun. Netw. 2023 (1) (2023) 7117863.
[16] X. Li, J. Xie, Q. Song, Y. Sang, Y. Zhang, S. Li, T. Zang, Let model keep evolving: Incremental learning for encrypted traffic classification, Comput. Secur. 137 (2024) 103624.
[17] C. Rong, G. Gou, M. Cui, G. Xiong, Z. Li, L. Guo, Malfinder: An ensemble learning-based framework for malicious traffic detection, in: 2020 IEEE Symposium on Computers and Communications, ISCC, IEEE, 2020, p. 7.
[18] B. Lu, N. Luktarhan, C. Ding, W. Zhang, ICLSTM: encrypted traffic service identification based on inception-LSTM neural network, Symmetry 13 (6) (2021) 1080.
[19] G. Ren, G. Cheng, N. Fu, Accurate encrypted malicious traffic identification via traffic interaction pattern using graph convolutional network, Appl. Sci. 13 (3) (2023) 1483.
[20] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, Soft Comput. 24 (3) (2020) 1999–2012.
[21] Z. Wang, M. Li, H. Ou, S. Pang, Z. Yue, A few-shot malicious encrypted traffic detection approach based on model-agnostic meta-learning, Secur. Commun. Netw. 2023 (1) (2023) 3629831.
[22] J. Yang, G. Liang, B. Li, G. Wen, T. Gao, A deep-learning-and reinforcement-learning-based system for encrypted network malicious traffic detection, Electron. Lett. 57 (9) (2021) 363–365.
[23] S. Soleymanpour, H. Sadr, M. Nazari Soleimandarabi, CSCNN: cost-sensitive convolutional neural network for encrypted traffic classification, Neural Process. Lett. 53 (5) (2021) 3497–3523.
[24] H. Wang, S. Mumtaz, H. Li, J. Liu, F. Yang, An identification strategy for unknown attack through the joint learning of space–time features, Future Gener. Comput. Syst. 117 (2021) 145–154.